# Visual Recognition Using Deep Learning Homework 1 Report

Pao-Hua Hsu

National Yang Ming Chiao Tung University, Hsinchu, Taiwan
Student ID: 314581025
paohuah.ii14@nycu.edu.tw

## 1 Introduction

This report follows the ECCV-style citation format [2] and summarizes my implementation for the NYCU Visual Recognition Using Deep Learning homework [4].

The foundation model I use is `timm/resnest200e.in1k` [6]. My final training pipeline uses both offline and online data augmentation. In the final version, each training image is converted into two images in total: one deterministic preprocessed copy and one additional augmented copy. The offline augmentation operations include `RandomHorizontalFlip`, `ColorJitter`, `RandomAffine`, and `GaussianBlur`.

During training, the online augmentation pipeline is lightweight. Each image is resized to $224 \times 224$, converted to a tensor, and normalized, and `RandomErasing` is applied with probability 0.2. In addition, `batch-level mixup/cutmix` is enabled during training to further improve generalization. `Label smoothing` is also used in the loss function. The final score is 0.93.

## 2 Method

### 2.1 Data Augmentation Strategy

To improve generalization, I experimented with several combinations of offline and online data augmentation. Offline augmentation was generated before training and saved to disk, while online augmentation was applied dynamically during training. In the early versions, I tested stronger augmentation settings that combined geometric and color transformations, including `RandomHorizontalFlip`, `ColorJitter`, `RandomAffine`, and `GaussianBlur`. I also explored different placements of mixup and random erasing, trying to determine whether they should be applied in the offline stage or only during training.

The augmentation settings evolved through several versions:

- V1: online augmentation + offline augmentation.
- V2: offline augmentation only.
- V3: fixed 1.5x offline generation policy with plain images, same-class mixup images, and random-erasing images.

– V4: final setting; 2x offline augmentation with geometric/color transforms, plus online `RandomErasing` and batch-level mixup/cutmix during training.

In earlier versions, I tested offline-only augmentation, online-only augmentation, and a combination of both. I also tried a fixed 1.5x offline policy, where each class was expanded with plain images, same-class mixup images, and random-erasing images. But the result show that if we don't use online augmentation the model will be overfitting execpt I use a very strong offline augmentation and highly dropout rate.

Because of the online augmentation is more important to prevent overfitting, the final version used in the experiment was a simpler 2x offline augmentation setting: each original training image was converted into one deterministic preprocessed copy and one additional augmented copy. In this final setting, the offline augmentation operations included `RandomHorizontalFlip`, `ColorJitter`, `RandomAffine`, and `GaussianBlur`.

For online augmentation in the final setting, each image was resized to $224 \times 224$, converted to a tensor, and normalized, with `RandomErasing` applied at training time. In addition, batch-level mixup/cutmix was enabled in the training loop to further improve robustness. Label smoothing was also applied in the training loss.

### 2.2 Model Variants

The main model is `timm/resnest200e.in1k` [6]. The model structure is shown in Fig. 1. Model parameters: 68.36M. This model achieved the best overall performance among my experiments, so most augmentation tuning was conducted on this backbone. ResNeSt is an improved variant of ResNeXt, and its full paper title is *Split-Attention Networks*. The key idea of ResNeSt is the split-attention block, which combines multi-path representation with channel-wise attention. In this design, the input feature maps are first divided into several cardinal groups, and each group is further split into multiple radix branches. The branch features are processed by convolution layers and then fused through split-attention before being aggregated with a residual shortcut connection. This design improves representation ability while keeping the computational overhead modest.

Besides ResNeSt-200e, I also tested `timm/res2next50.in1k` [3]. The purpose of this experiment was to compare whether a different architecture could benefit from the same augmentation strategy. Although Res2NeXt50 also worked reasonably well, its performance was lower than the best ResNeSt-200e setting, so the final submission was based on ResNeSt-200e.

### 2.3 Hyperparameters

I used Optuna [1] to search for the best hyperparameters. Base on data augmentation setting V3, and ResNeSt-200e backbone. The best hyperparameters are as follows:

– Batch size: 64

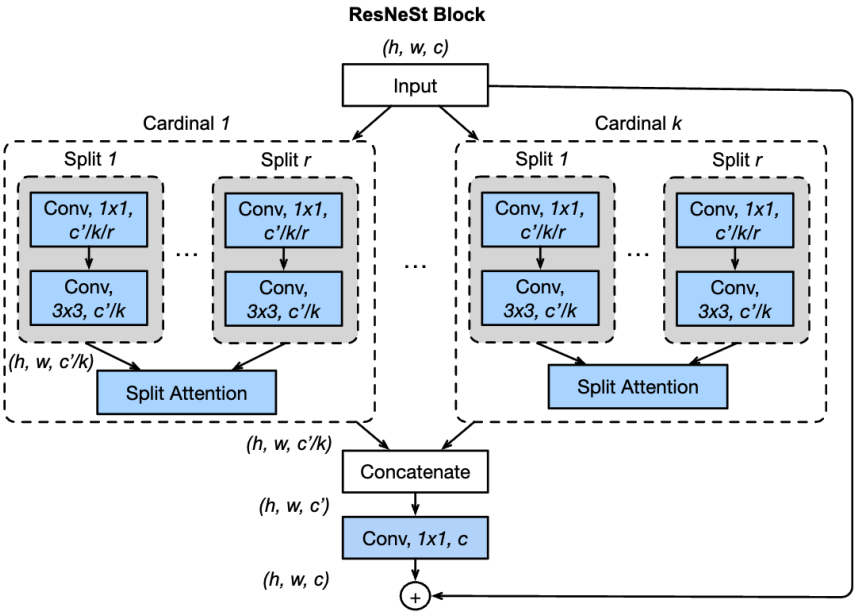


**Fig. 1:** Structure of the ResNeSt backbone used in this work.

- Learning rate of classifier head in stage 1: 0.0019440246330402697
- Learning rate of backbone in stage 2: 6.062319554690634e-5
- Learning rate of classifier head in stage 2: 0.0004223368566223033
- Dropout rate: 0.6142405973299517
- Weight decay: 6.506552238283957e-5
- Label smoothing: 0.19013432446567302

## 3 Results

The experimental results are summarized in Tab. 1. I omit most intermediate figures here and keep only the validation scores for clearer comparison.

For the ResNeSt-200e series, `resnest200e_V1` uses both online and offline data augmentation and achieves a score of 0.90. `resnest200e_V2` removes online augmentation and keeps only offline augmentation, resulting in a score of 0.88. Although the training accuracy increased to about 99%, the validation result was worse, indicating that the model suffered from overfitting without online augmentation. `resnest200e_V3` online and offline augmentation were both used. And the training strategy changed to 2 stages: first train the classifier head for 5 epochs, then fine-tune the whole model for 45 epochs.The result is reaching 0.91. `resnest200e_V4` updates the augmentation design, but its performance is slightly lower than V3, with a score of 0.90.

**Table 1:** Summary of validation scores for different models and augmentation settings.

| Model / Version | Backbone | Score |
|---|---|---|
| `resnest200e_V1` | ResNeSt-200e | 0.90 |
| `resnest200e_V2` | ResNeSt-200e | 0.88 |
| `resnest200e_V3` | ResNeSt-200e | 0.91 |
| `resnest200e_V4` | ResNeSt-200e | 0.90 |
| `resnest200e_optuna_V2` | ResNeSt-200e | 0.93 |
| `resnest200e_optuna_V3` | ResNeSt-200e | 0.93 |
| `res2next50_with_V3` | Res2NeXt-50 | 0.86 |
| `dinov2-giant-imagenet1k-1-layer` | DINOv2 | 0.94 |
| `dinov2-base-imagenet1k-1-layer` | DINOv2 | 0.95 |

`resnest200e_optuna_V2` is based on `timm/resnest200e.in1k` [6] and data augmentation V3 with Optuna-tuned hyperparameters [1]. The model is trained in two stages: 5 epochs for classifier-head training and 45 epochs for full fine-tuning, and the detailed hyperparameter setting is provided in Sec. 2.3.

`resnest200e_optuna_V3` further avoids using the same augmentation strategy in both offline and online stages. The offline dataset is expanded to 2x of the original size with geometric and color augmentation, while random erasing and batch-level mixup/cutmix are kept in the online training stage. This version achieves the best performance among my ResNeSt-200e experiments, with a score of 0.93.

For Res2NeXt-50, the `res2next50_with_V3` experiment applies the V3-style augmentation setting to `res2next50` [3] and achieves a score of 0.86.

For other backbone experiments, `dinov2-giant-imagenet1k-1-layer` reaches 0.94, while `dinov2-base-imagenet1k-1-layer` reaches 0.95. The stronger model based on DINOv2 [5] achieves better performance than the ResNeSt-200e backbone. Additionally, when I used DINOv2, the augmentation strategy was similar to the final ResNeSt-200e setting. DINOv2 achieves 0.95 quite easily with that augmentation strategy, which further confirms the effectiveness of the final augmentation design.

## References

1. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2019)
2. European Conference on Computer Vision (ECCV): Author guidelines for eccv submission. `https://eccv.ecva.net/` (2026)
3. Gao, S.H., Cheng, M.M., Zhao, K., Zhang, X.Y., Yang, M.H., Torr, P.H.S.: Res2net: A new multi-scale backbone architecture. arXiv preprint arXiv:1904.01169 (2019)
4. Lin, Y.Y.: Selected topics in visual recognition using deep learning. `https://docs.google.com/presentation/d/1MAi-JWLOaFQlF4VXxOKkpxdGPiTalYbnjc7Km5TpFuM/edit?usp=share_link` (2026), national Yang Ming Chiao Tung University course slides, Spring 2026

5. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.Y., Li, S.W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Bojanowski, P., Joulin, A.: Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193 (2023)
6. Zhang, H., Wu, C., Zhang, Z., Zhu, Y., Lin, H., Sun, Y., He, T., Mueller, J., Manmatha, R., Li, M., Smola, A.: Resnest: Split-attention networks. arXiv preprint arXiv:2004.08955 (2020)